

Introduction to using XPath in JavaScript

This document describes the interface for using XPath in JavaScript internally, in extensions, and from websites. Mozilla implements a fair amount of the DOM 3 XPath. Which means that XPath expressions can be run against both HTML and XML documents.

The main interface to using XPath is the evaluate function of the document object.

document.evaluate

This method evaluates XPath expressions against an XML based document (including HTML documents), and returns a `XPathResult` object, which can be a single node or a set of nodes. The existing documentation for this method is located at DOM:document.evaluate, but it is rather sparse for our needs at the moment; a more comprehensive examination will be given below.

```
var xpathResult = document.evaluate( xpathExpression, contextNode, namespaceResolver, resultType, result );
```

Parameters

The evaluate function takes a total of five parameters:

- **xpathExpression**: A string containing the XPath expression to be evaluated.
- **contextNode**: A node in the document against which the `xpathExpression` should be evaluated, including any and all of its child nodes. The document node is the most commonly used.
- **namespaceResolver**: A function that will be passed any namespace prefixes contained within `xpathExpression` which returns a string representing the namespace URI associated with that prefix. This enables conversion between the prefixes used in the XPath expressions and the possibly different prefixes used in the document. The function can be either:
 - Created by using the `createNSResolver` method of a `XPathEvaluator` object. You should use this virtually all of the time.
 - `null`, which can be used for HTML documents or when no namespace prefixes are used. Note that, if the `xpathExpression` contains a namespace prefix, this will result in a `DOMException` being thrown with the code `NAMESPACE_ERR`.
 - A custom user-defined function. See the Using a User Defined Namespace Resolver section in the appendix for details.
- **resultType**: A constant that specifies the desired result type to be returned as a result of the evaluation. The most commonly passed constant is `XPathResult.ANY_TYPE` which will return the results of the XPath expression as the most natural type. There is a section in the appendix which contains a full list of the available constants. They are explained below in the section "Specifying the Return Type."
- **result**: If an existing `XPathResult` object is specified, it will be reused to return the results. Specifying `null` will create a new `XPathResult` object.

Return Value

Returns `xpathResult`, which is an `XPathResult` object of the type specified in the `resultType` parameter. The `XPathResult` Interface is defined here.

Implementing a Default Namespace Resolver

We create a namespace resolver using the `createNSResolver` method of the document object.

```
var nsResolver = document.createNSResolver( contextNode.ownerDocument == null ? contextNode.documentElement : contextNode.ownerDocument.documentElement );
```

And then pass `document.evaluate`, the `nsResolver` variable as the `namespaceResolver` parameter.

Note: XPath defines QNames without a prefix to match only elements in the null namespace. There is no way in XPath to pick up the default namespace as applied to a regular element reference (e.g., `p[@id='_myid']` for `xmlns='http://www.w3.org/1999/xhtml'`). To match default elements in a non-null namespace, you either have to refer to a particular element using a form such as `*/div/div[3]/p[3]/code[3]/span, line 1, column 70: ")"` expected[@id='_myid'] (this approach works well for dynamic XPath's where the namespaces might not be known) or use prefixed name tests, and create a namespace resolver mapping the prefix to the namespace. Read more on how to create a user defined namespace resolver, if you wish to take the latter approach.

Notes

Adapts any DOM node to resolve namespaces so that an XPath expression can be easily evaluated relative to the context of the node where it appeared within the document. This adapter works like the DOM Level 3 method `lookupNamespaceURI` on nodes in resolving the `namespaceURI` from a given prefix using the current information available in the node's hierarchy at the time `lookupNamespaceURI` is called. Also correctly resolves the implicit `xml` prefix.

Specifying the Return Type

The returned variable `xpathResult` from `document.evaluate` can either be composed of individual nodes (simple types), or a collection of nodes (node-set types).

Simple Types

When the desired result type in `resultType` is specified as either:

- `NUMBER_TYPE` - a double
- `STRING_TYPE` - a string
- `BOOLEAN_TYPE` - a boolean

We obtain the returned value of the expression by accessing the following properties respectively of the `XPathResult` object.

- `numberValue`
- `stringValue`
- `booleanValue`

Example

The following uses the XPath expression `count(//p)` to obtain the number of `<p>` elements in a HTML document:

```
var paragraphCount = document.evaluate( 'count(//p)', document, null, XPathResult.ANY_TYPE, null );
alert( 'This document contains ' + paragraphCount.numberValue + ' paragraph elements' );
```

Although Javascript allows us to convert the number to a string for display, the XPath interface will not automatically convert the numerical result if the `stringValue` property is requested, so the following code will **'not'** work:

```
var paragraphCount = document.evaluate('count(//p)', document, null, XPathResult.ANY_TYPE, null );
alert( 'This document contains ' + paragraphCount.stringValue + ' paragraph elements' );
```

Instead it will return an exception with the code `NS_DOM_TYPE_ERROR`.

Node-Set Types

The `XPathResult` object allows node-sets to be returned in 3 principal different types:

- Iterators
- Snapshots
- First Nodes

Iterators

When the specified result type in the `resultType` parameter is either:

- `UNORDERED_NODE_ITERATOR_TYPE`
- `ORDERED_NODE_ITERATOR_TYPE`

The `XPathResult` object returned is a node-set of matched nodes which will behave as an iterator, allowing us to access the individual nodes contained by using the `iterateNext()` method of the `XPathResult`.

Once we have iterated over all of the individual matched nodes, `iterateNext()` will return `null`.

Note however, that if the document is mutated (the document tree is modified) between iterations that will invalidate the iteration and the `invalidIteratorState` property of `XPathResult` is set to `true`, and a `NS_ERROR_DOM_INVALID_STATE_ERR` exception is thrown.

Iterator Example

```
var iterator = document.evaluate('//phoneNumber', documentNode, null, XPathResult.UNORDERED_NODE_ITERATOR_TYPE, null );
try {
    var thisNode = iterator.iterateNext();

    while (thisNode) {
        alert( thisNode.textContent );
        thisNode = iterator.iterateNext();
    }
} catch (e) {
    dump( 'Error: Document tree modified during iteration ' + e );
}
```

Snapshots

When the specified result type in the `resultType` parameter is either:

- `UNORDERED_NODE_SNAPSHOT_TYPE`
- `ORDERED_NODE_SNAPSHOT_TYPE`

The `XPathResult` object returned is a static node-set of matched nodes, which allows us to access each node through the `snapshotItem(itemNumber)` method of the `XPathResult` object, where `itemNumber` is the index of the node to be retrieved. The total number of nodes contained can be accessed through the `snapshotLength` property.

Snapshots do not change with document mutations, so unlike the iterators the snapshot does not become invalid, but it may not correspond to the current document, for example the nodes may have been moved, it might contain nodes that no longer exist, or new nodes could have been added.

Snapshot Example

```
var nodesSnapshot = document.evaluate('//phoneNumber', documentNode, null, XPathResult.ORDERED_NODE_SNAPSHOT_TYPE, null );
for ( var i=0 ; i < nodesSnapshot.snapshotLength; i++ )
{
    dump( nodesSnapshot.snapshotItem(i).textContent );
}
```

First Node

When the specified result type in the `resultType` parameter is either:

- `ANY_UNORDERED_NODE_TYPE`
- `FIRST_ORDERED_NODE_TYPE`

The `XPathResult` object returned is only the first found node that matched the XPath expression. This can be accessed through the `singleNodeValue` property of the `XPathResult` object. This will be `null` if the node set is empty.

Note that, for the unordered subtype the single node returned might not be the first in document order, but for the ordered subtype you are guaranteed to get the first matched node in the document order.

First Node Example

```
var firstPhoneNumber = document.evaluate('//phoneNumber', documentNode, null, XPathResult.FIRST_ORDERED_NODE_TYPE, null );
dump( 'The first phone number found is ' + firstPhoneNumber.singleNodeValue.textContent );
```

The ANY_TYPE Constant

When the result type in the `resultType` parameter is specified as `ANY_TYPE`, the `XPathResult` object returned, will be whatever type that naturally results from the evaluation of the expression.

It could be any of the simple types (`NUMBER_TYPE`, `STRING_TYPE`, `BOOLEAN_TYPE`), **but**, if the returned result type is a node-set then it will **only** be an `UNORDERED_NODE_ITERATOR_TYPE`.

To determine that type after evaluation, we use the `resultType` property of the `XPathResult` object. The constant values of this property are defined in the appendix.

Examples

Within an HTML Document

The following code is intended to be placed in any JavaScript fragment within or linked to the HTML document against which the XPath expression is to be evaluated.

To extract all the `<h2>` heading elements in an HTML document using XPath, the `xpathExpression` is simply `'//h2'`. Where, `//` is the Recursive Descent Operator that matches elements with the nodeName `h2` anywhere in the document tree. The full code for this is:

```
var headings = document.evaluate('//h2', document, null, XPathResult.ANY_TYPE, null );
```

Notice that, since HTML does not have namespaces, we have passed `null` for the `namespaceResolver` parameter.

Since we wish to search over the entire document for the headings, we have used the document object itself as the `contextNode`.

The result of this expression is an `XPathResult` object. If we wish to know the type of result returned, we may evaluate the `resultType` property of the returned object. In this case, that will evaluate to 4, an `UNORDERED_NODE_ITERATOR_TYPE`. This is the default return type when the result of the XPath expression is a node set. It provides access to a single node at a time and may not return nodes in a particular order. To access the returned nodes, we use the `iterateNext()` method of the returned object:

```
var thisHeading = headings.iterateNext();

var alertText = 'Level 2 headings in this document are:\n'

while (thisHeading) {
    alertText += thisHeading.textContent + '\n';
    thisHeading = headings.iterateNext();
}
```

Once we iterate to a node, we have access to all the standard DOM interfaces on that node. After iterating through all the `h2` elements returned from our expression, any further calls to `iterateNext()` will return `null`.

Evaluating against an XML document within an Extension

The following uses an XML document located at `chrome://yourextension/content/peopleDB.xml` as an example.

```
<?xml version="1.0"?>
<people xmlns:xul = "http://www.mozilla.org/keymaster/gatekeeper/there.is.only.xul" >
  <person>
    <name first="george" last="bush" />
    <address street="1600 pennsylvania avenue" city="washington" country="usa"/>
    <phoneNumber>202-456-1111</phoneNumber>
  </person>
```

```

<person>
  <name first="tony" last="blair" />
  <address street="10 downing street" city="london" country="uk"/>
  <phoneNumber>020 7925 0918</phoneNumber>
</person>
</people>

```

To make the contents of the XML document available within the extension, we create an `XMLHttpRequest` object to load the document synchronously, the variable `xmlDoc` will contain the document as an `XMLDocument` object against which we can use the `evaluate` method

JavaScript used in the extensions `xul/js` documents.

```

var req = new XMLHttpRequest();

req.open("GET", "chrome://yourextension/content/peopleDB.xml", false);
req.send(null);

var xmlDoc = req.responseXML;

var nsResolver = xmlDoc.createNSResolver( xmlDoc.ownerDocument == null ? xmlDoc.documentElement : xmlDoc.ownerDocument.documentElement );

var personIterator = xmlDoc.evaluate('//person', xmlDoc, nsResolver, XPathResult.ANY_TYPE, null );

```

Appendix

Implementing a User Defined Namespace Resolver

This is an example for illustration only. This function will need to take namespace prefixes from the `xpathExpression` and return the URI that corresponds to that prefix. For example, the expression:

```
'//xhtml:td/mathml:math'
```

will select all MathML expressions that are the children of (X)HTML table data cell elements.

In order to associate the `'mathml:'` prefix with the namespace URI `'http://www.w3.org/1998/Math/MathML'` and `'xhtml:'` with the URI `'http://www.w3.org/1999/xhtml'` we provide a function:

```

function nsResolver(prefix) {
  var ns = {
    'xhtml' : 'http://www.w3.org/1999/xhtml',
    'mathml': 'http://www.w3.org/1998/Math/MathML'
  };
  return ns[prefix] || null;
}

```

Our call to `document.evaluate` would then look like:

```
document.evaluate( '//xhtml:td/mathml:math', document, nsResolver, XPathResult.ANY_TYPE, null );
```

Implementing a default namespace for XML documents

As noted in the `#Implementing a Default Namespace Resolver` previously, the default resolver does not handle the default namespace for XML documents. For example with this document:

```
<?xml version="1.0" encoding="UTF-8"?>
<feed xmlns="http://www.w3.org/2005/Atom">
  <entry />
  <entry />
  <entry />
</feed>
```

`doc.evaluate('//entry', doc, nsResolver, XPathResult.ANY_TYPE, null)` will return an empty set, where `nsResolver` is the resolver returned by `createNSResolver`. Passing a `null` resolver doesn't work any better, either.

One possible workaround is to create a custom resolver that returns the correct default namespace (the Atom namespace in this case). Note that you still have to use some namespace prefix in your XPath expression, so that the resolver function will be able to change it to your required namespace. E.g.:

```
function resolver() {
  return 'http://www.w3.org/2005/Atom';
}
doc.evaluate('//myns:entry', doc, resolver, XPathResult.ANY_TYPE, null)
```

Note that a more complex resolver will be required if the document uses multiple namespaces.

An approach which might work better (and allow namespaces not to be known ahead of time) is described in the next section.

Using XPath functions to reference elements with a default namespace

Another approach to match default elements in a non-null namespace (and one which works well for dynamic XPath expressions where the namespaces might not be known), involves referring to a particular element using a form such as `*/div[3]/div[3]/p/code/span, line 1, column 70: ")" expected[@id='_myid']`. This circumvents the problem of an XPath query not being able to detect the default namespace on a regularly labeled element.

Getting specifically namespaced elements and attributes regardless of prefix

If one wishes to provide flexibility in namespaces (as they are intended) by not necessarily requiring a particular prefix to be used when finding a namespaced element or attribute, one must use special techniques.

While one can adapt the approach in the above section to test for namespaced elements regardless of the prefix chosen (using `local-name()` in combination with `namespace-uri()` instead of `name()`), a more challenging situation occurs, however, if one wishes to grab an element with a particular namespaced attribute in a predicate (given the absence of implementation-independent variables in XPath 1.0).

For example, one might try (incorrectly) to grab an element with a namespaced attribute as follows:
`var xpathlink = 'someElements/div[3]/div[4]/p[3]/code/span, line 1, column 104: ")" expected';`

This could inadvertently grab some elements if one of its attributes existed that had a local name of "href", but it was a different attribute which had the targeted (XLink) namespace (instead of @href).

In order to accurately grab elements with the XLink @href attribute (without also being confined to predefined prefixes in a namespace resolver), one could obtain them as follows:

```
var xpathEls = 'someElements[@{local-name() = "href" and namespace-uri() = "http://www.w3.org/1999/xlink"}]'; // Grabs elements with any single attribute that has both the local name 'href' and the XLink namespace
var thisLevel = xml.evaluate(xpathEls, xml, null, XPathResult.ANY_TYPE, null);
var thisItemEl = thisLevel.iterateNext();
```

XPathResult Defined Constants

Result Type Defined Constant	Value	Description
ANY_TYPE	0	A result set containing whatever type naturally results from evaluation of the expression. Note that if the result is a node-set then <code>UNORDERED_NODE_ITERATOR_TYPE</code> is always the resulting type.
NUMBER_TYPE	1	A result containing a single number. This is useful for example, in an XPath expression using the <code>count ()</code> function.
STRING_TYPE	2	A result containing a single string.
BOOLEAN_TYPE	3	A result containing a single boolean value. This is useful for example, in an XPath expression using the <code>not ()</code> function.
UNORDERED_NODE_ITERATOR_TYPE	4	A result node-set containing all the nodes matching the expression. The nodes may not necessarily be in the same order that they appear in the document.
ORDERED_NODE_ITERATOR_TYPE	5	A result node-set containing all the nodes matching the expression. The nodes in the result set are in the same order that they appear in the document.
UNORDERED_NODE_SNAPSHOT_TYPE	6	A result node-set containing snapshots of all the nodes matching the expression. The nodes may not necessarily be in the same order that they appear in the document.
ORDERED_NODE_SNAPSHOT_TYPE	7	A result node-set containing snapshots of all the nodes matching the expression. The nodes in the result set are in the same order that they appear in the document.
ANY_UNORDERED_NODE_TYPE	8	A result node-set containing any single node that matches the expression. The node is not necessarily the first node in the document that matches the expression.
FIRST_ORDERED_NODE_TYPE	9	A result node-set containing the first node in the document that matches the expression.

See also

- [Using XPath](#)

Original Document Information

- Based Upon Original Document Mozilla XPath Tutorial
- Original Source Author: James Graham.
- Other Contributors: James Thompson.
- Last Updated Date: 2006-3-25.

Attached Files

Attached Images